

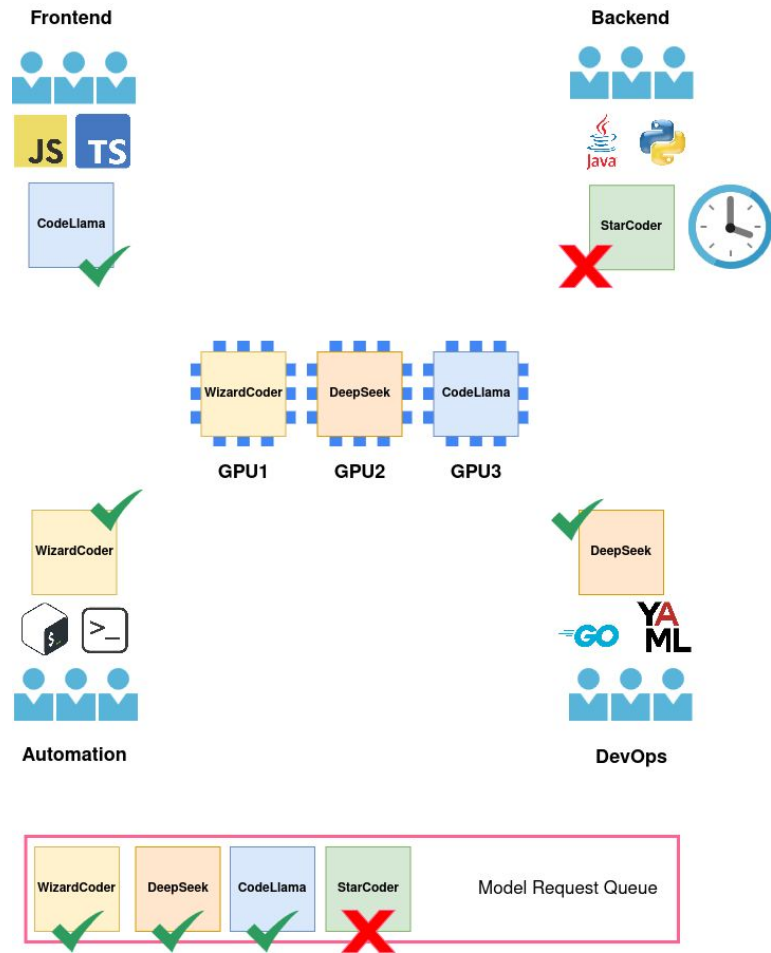
Context Aware CodeLLM Eviction

Kishanthan Thangarajah, Boyuan Chen, Shi Chang, Ahmed E. Hassan
Centre for Software Excellence, Huawei Canada



The Challenge of Self-Hosting CodeLLMs

- **Diverse Enterprise Ecosystems**
 - Enterprise products span multiple domains and teams (front-end, back-end, DevOps, and automation groups), each using different languages.
 - CodeLLMs boost developer productivity by automating routine coding, refactoring, and code review tasks.
- **Need for Self-Hosting CodeLLMs**
 - Data privacy & security → Keep proprietary source code and IP within the organization; meet compliance (e.g., GDPR).
 - Low-latency developer experience → Eliminate network delays to external APIs for real-time IDE completions.
- **Implications for Self-Hosting CodeLLMs**
 - CodeLLMs specialize by language and task → some fine-tuned for Python or Java, others for reasoning or debugging.
 - GPU memory is limited → can't keep all these models active.
 - Frequent load/unload cycles between projects cause cold-start latency.
 - Developers expect < 200 ms completions, but cold starts take 2–3 s.



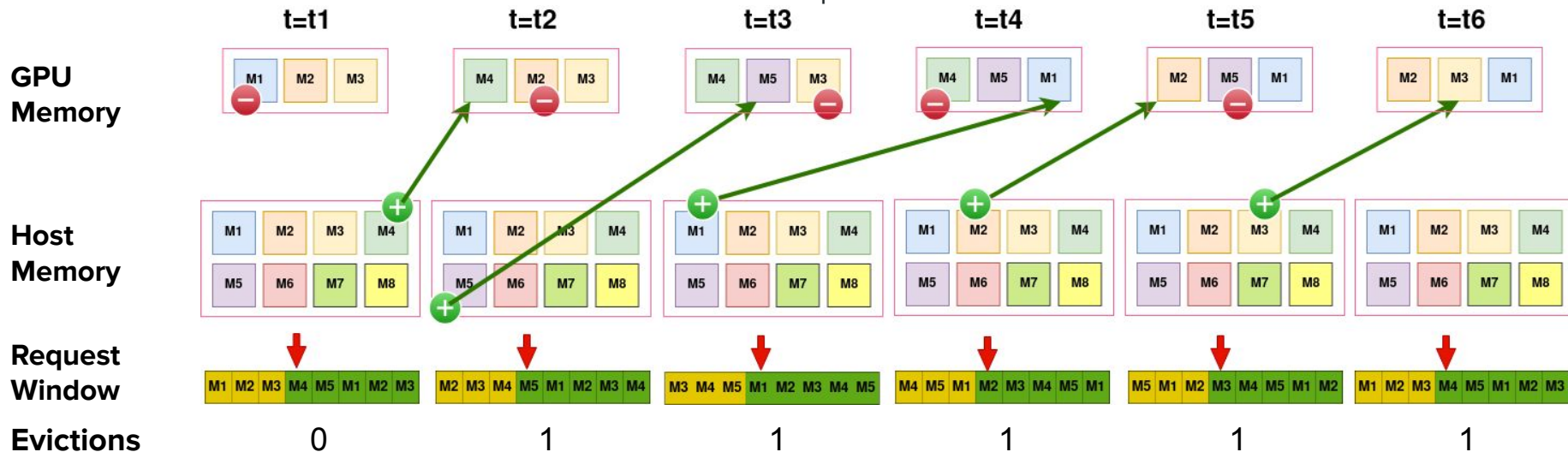
LRU Eviction is Not Enough

- **How Today's Systems Work**

- Systems like Ray Serve, Ollama, ModelMesh, and FaaSSwap use **LRU** for model eviction.
- LRU decides based only on how **recently** a model was used.

- **Why This Fails in Practice**

- Ignores large differences in model load times (e.g., a 70 B model $\neq$ a 1 B model).
- Ignores task type: some tasks are latency-critical (code-completions) while others can wait (code-reasoning).
- Ignores future demand from queued or predictable requests.



LRU based approach incurs **5 model evictions** based on the least recently used timestamp.

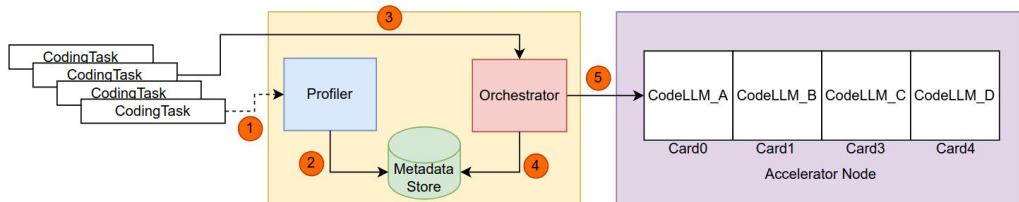
Making Model Evictions Smarter: Introducing CACE

CACE : Context-Aware CodeLLM Eviction.

Instead of a single-factor LRU, CACE computes a multi-factor Eviction Score that weighs:

- **Recency** – how recently the model was used (P1)
- **Reload Cost** – model's load time / size (P2)
- **Future Demand** – predicted next request window (P3)
- **Task Criticality** – latency sensitivity (TTFT vs E2E) (P4)

Models with the highest EvictionScore are least valuable and selected for eviction.

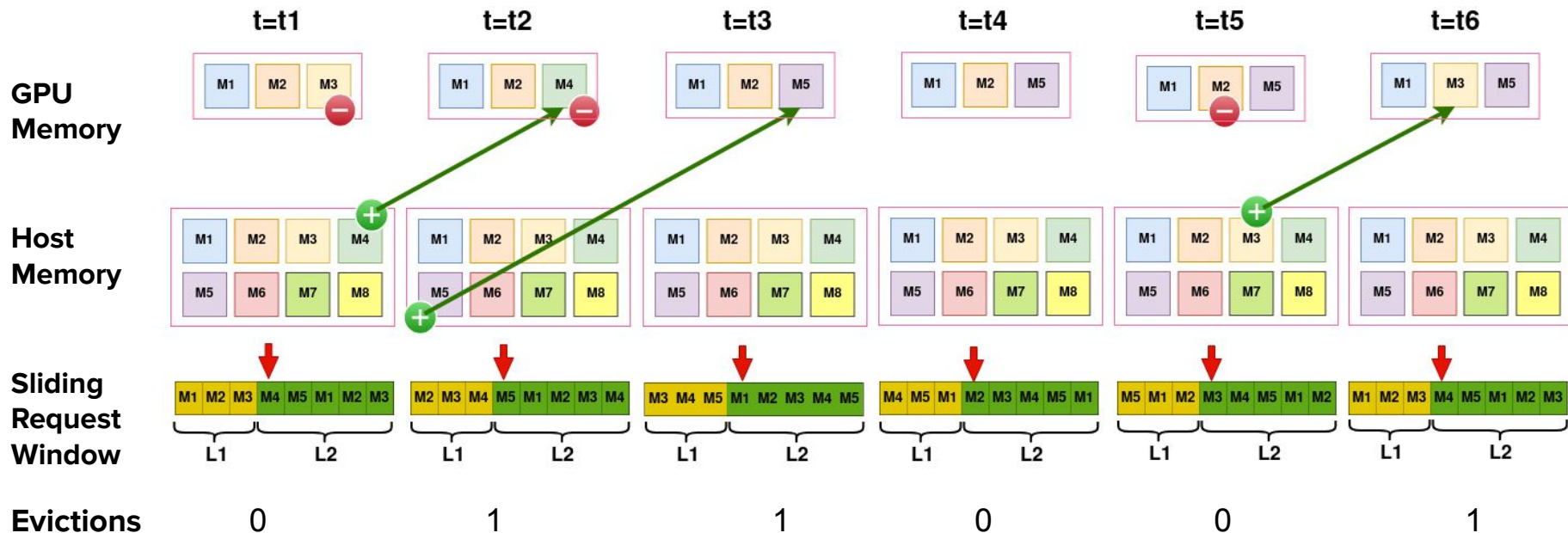


Let:

- t : last used time of the model in L1 (timestamp),
- l : model load time (in seconds),
- w : sliding window length,
- i : index position of the model in L2,
- o : expected number of output tokens,
- w_1 : weighting factor for output tokens.

$$\text{EvictionScore} = \underbrace{\frac{1}{1 + \ln t}}_{P1} + \underbrace{\frac{1}{1 + \frac{l}{100}}}_{P2} + \underbrace{\frac{i}{w}}_{P3} + \underbrace{w_1 \cdot o}_{P4}$$

How CACE Works in Practice



CACE incurs only **3 model evictions** as opposed to LRU.

At t_1 , CACE considers both past usage (L1) and predicted future demand (L2) and decides to evict M3. Similarly at t_2 , it evicts M4.

No evictions occur at t_3 and t_4 , demonstrating stable model retention.

CACE Delivers Consistent Gains with Evaluation

Benchmarks: McEval (multilingual code completion) & CRUXEVAL-X (multilingual code reasoning). Completion is TTFT-sensitive; reasoning is E2E-sensitive.

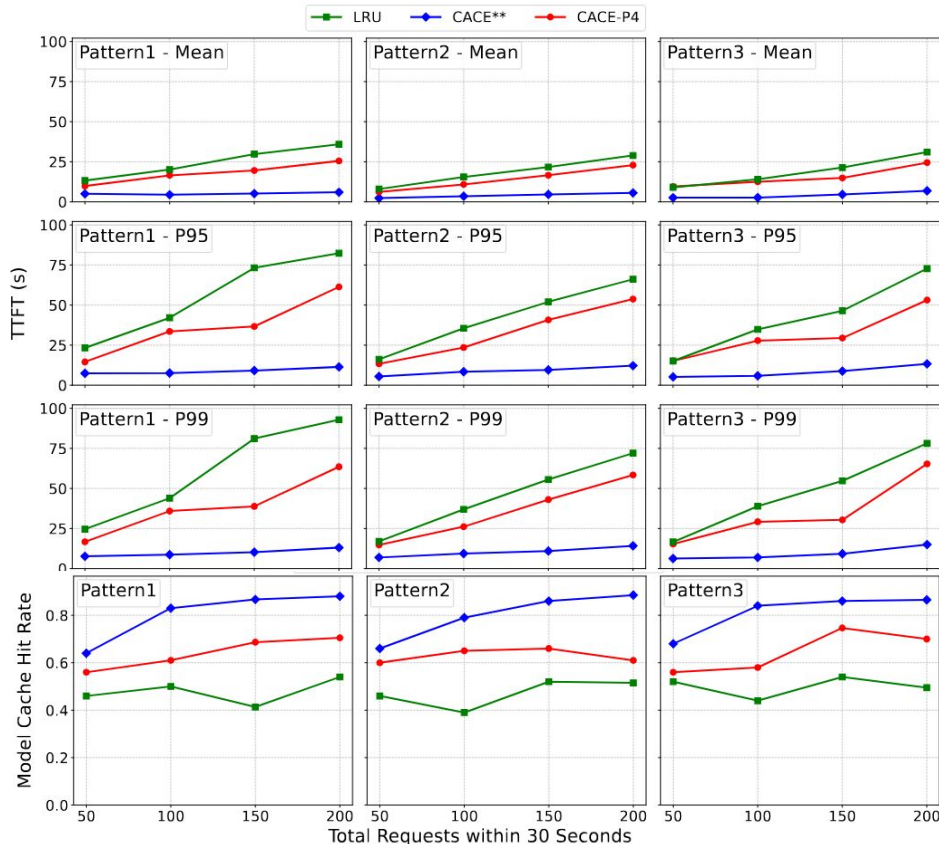
Languages: 8 total, Java, Python, C++, C, Go, Rust, C#, JavaScript; small (~500M) models per language for completion, large (~7B) for reasoning.

Workload patterns (30s Poisson arrivals):

- Pattern 1: Uniform (50% completion / 50% reasoning across 8 languages).
- Pattern 2: IDE-Heavy (70% completion / 30% reasoning).
- Pattern 3: Language Popularity Skew (70/30 task split with 20% each to Java, Python, C++, JS; rest to Go, Rust, C, C#).

Metrics:

- TTFT: time-to-first-token (completion).
- E2E latency: end-to-end (reasoning).
- Model Cache Hit Rate and Eviction/Load Overhead.



CACE outperforms LRU by cutting **TTFT** by up to **70 %**, reducing **E2E** latency by up to **37 %**, and boosting **model cache hit** rate to **0.86** (= +55 %). It also halves reload overhead, driven by task awareness and demand forecasting.

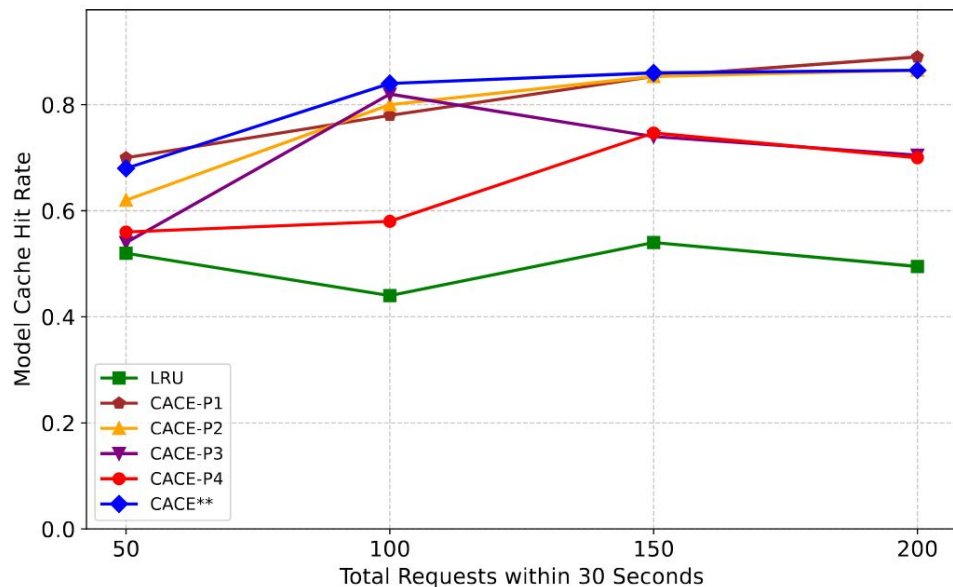
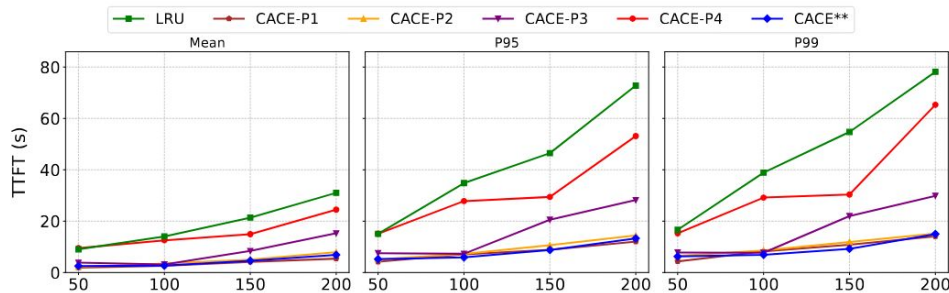
Which Eviction Factors Matter Most? - Ablation Insights

P4 – Task Criticality	▲ Most critical factor → Removing it causes P95 TTFT ↑ 72 %, E2E ↑ 47 %, Hit Rate ↓ 24 %
P3 – Future Demand	⚡ Second most influential → Removing it reduces Hit Rate by 22 % and increases P95 TTFT by 15 s.
P2 – Reload Cost	⚙️ Helps reduce unnecessary reloads (~10 %)
P1 – Recency	⚙️ Minor effect → Small latency increase (~5 %)

CACE's effectiveness stems from **P4 (Task Criticality)** and **P3 (Future Demand)**; **P1 (Recency)** and **P2 (Reload Cost)** contribute smaller incremental benefits.

Effective eviction = prioritizing critical, soon-to-be-used models.

Future work can explore adaptive weighting for even finer control under dynamic workloads.



Q & A

Join us at the Industry Showcase Poster Session
to learn more about CACE.